« Linux Capabilities »

Jean-Marc Pouchoulon

Mai 2026

Vous travaillerez dans une VM Debian à la dernière version stable.

Utilisez le compte `root`.

Après les « NameSpaces » et les « CGroups », c'est la troisième brique de la **conteneurisation**.

Les *capabilities* s'appliquent aux **threads** (processus légers Linux) et aux fichiers binaires.

Un processus ou un ensemble de processus doit avoir ce qu'il lui faut comme *capabilities* afin de fonctionner, mais pas plus, sous peine de servir de passerelle à un attaquant.

Les *capabilities* peuvent aussi servir afin de donner des capacités à un programme sans avoir besoin de l'ensemble des droits du compte `root` ou d'avoir à positionner le bit **SUID**.

Exemple : le bit SUID et ses limites

Le bit SUID (Set User ID) permet à un programme de s'exécuter avec les droits de son *propriétaire* plutôt que ceux de l'utilisateur qui le lance. L'exemple classique est `passwd` :

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 68208 ...  /usr/bin/passwd
```

Le `s` à la place du `x` du propriétaire indique le bit SUID. Quand `student` lance `passwd`, le processus s'exécute avec les droits de `root` pour pouvoir écrire dans `/etc/shadow`. Le problème : si un attaquant exploite une faille dans ce binaire, il obtient **l'intégralité** des droits `root`. Les *capabilities* permettent d'éviter cela :

```
# Retirer le bit SUID et accorder uniquement la capability nécessaire
sudo chmod u-s /usr/bin/passwd
sudo setcap cap_dac_override+ep /usr/bin/passwd
```

Ainsi, une faille dans `passwd` ne donnerait accès qu'à `CAP_DAC_OVERRIDE` et non à l'ensemble des privileges root : c'est le principe du **moindre privilège**.

Les systèmes de fichiers les plus courants implémentent les *capabilities* dans les inodes.

Il existe deux bibliothèques libpcap pour travailler avec les *capabilities*. : In`libcap2-bin` et `libcap-ng-utils`.

Ces packages vous donneront accès aux commandes pour afficher ou modifier les *capabilities* :

- `capsh`
- `getcap`
- `setcap`
- `filecap`
- `pscap` (liste les capabilities des processus en cours d'exécution)

1 Lecture des capabilities

1. Quelles sont les pouvoirs des "capabilities" suivantes: CAP_NET_ADMIN,CAP_NET_RAW, CAP_NET_BIND_SERVICE, CAP_SYS_ADMIN.
2. Créer une machine virtuelle Debian avec incus pour test (virtualbox ne doit pas être lancé):

```
systemctl start incus
cd /tmp
git clone https://github.com/pushou/incusprofile
cd incusprofile
incus profile create deb13-profile < deb13.profile
incus launch images:debian/13/cloud --profile deb13-profile --vm deb13
incus shell deb13
```

3. Retrouvez les capabilities de votre kernel à l'aide la commande "firejail --debug-caps".
4. Listez tous les programmes de votre machine avec "capabilities" avec la commande suivante:

```
getcap -r / 2>/dev/null
```

5. À l'aide de la commande `getcap` retrouvez les capabilities attachées au programme `mtr-packet`.
6. Utilisez `pscap` pour lister les capabilities des processus en cours d'exécution. Que remarquez-vous ?
7. Sous l'utilisateur `student` (non privilégié), visualisez les capabilities de votre processus bash dans `/proc` à l'aide de la commande suivante :

```
cat /proc/$$/status | egrep "^Cap"
```

8. A quoi correspondent CapInh,CapPrm,CapEff ?
9. Que donne la commande précédente avec un processus bash sous `root` ? Comparez les valeurs de `CapPrm`, `CapEff`, `CapBnd` et `CapAmb` entre les deux utilisateurs.
10. Utilisez `capsh \detokenize{--}decode=<valeur>` pour décoder les *effective capabilities* (`CapEff`) sous `student` et `root`.
11. Inspectez aussi les champs `CapBnd` (Bounding set) et `CapAmb` (Ambient) pour les deux utilisateurs et expliquez leur rôle.

2 Modifications des capabilities

1. Lancez la commande `python3 -m http.server <port>` avec comme port 9000 puis 80. Que se passe-t-il avec le port 80 ? Expliquez.
2. Donnez la *capability* permettant de se lier (*bind*) sur le port 80 en utilisant `setcap`.
3. Vérifiez avec `getcap` que la capability a bien été acquise sur le binaire.
4. Supprimez ensuite la capability du binaire avec `sudo setcap -r /usr/bin/python3.X` et vérifiez qu'elle a bien été retirée. (le X est la version de python3 installée sur votre machine, par exemple 3.13).s
5. Sous `root`, supprimez toutes les capabilities de votre processus bash et vérifiez que vous ne pouvez plus rien faire (`ping`, `tcpdump`...).

```
capsh --drop=all --secbits=1 --
```

*Note : l'option `-secbits=1` active le bit **SECBIT_NOROOT** qui empêche le processus de récupérer des capabilities via `execve` d'un binaire set-uid-root. Le double tiret final « - » lance un nouveau shell avec ces restrictions.*

3 Capabilities et conteneurs Docker

1. Par défaut, Docker supprime un grand nombre de capabilities lorsqu'il lance un conteneur. Lancez la commande suivante et observez les capabilities disponibles dans le conteneur :

   ```
   docker run --rm alpine sh -c "cat /proc/1/status | grep Cap"
   ```

   Décodez les valeurs avec `capsh -decode=<valeur>` et comparez-les aux capabilities d'un processus root sur l'hôte.
2. Lancez un conteneur en ajoutant la capability `CAP_NET_ADMIN` puis vérifiez son effet :

   ```
   docker run --rm --cap-add NET_ADMIN alpine sh -c "cat /proc/1/status | grep Cap"
   ```

3. Lancez un conteneur en supprimant **toutes** les capabilities par défaut, puis ajoutez uniquement `CAP_CHOWN` :

   ```
   docker run --rm --cap-drop ALL --cap-add CHOWN alpine sh -c \
     "cat /proc/1/status | grep Cap && capsh --decode=$(grep CapEff /proc/1/status | awk '{print $2}')"
   ```

   Que pouvez-vous en conclure sur le principe du **moindre privilège** dans les conteneurs ?

4 IMPERATIF à la fin du tp

```
incus rm -f deb13
```